

Arquitetura Spring Boot

Java, API REST e MVC

Guia pratico com exemplos, analogias e trilha de estudos

Feito para estudar arquitetura de projetos Java com Spring Boot.

Foco: APIs REST, MVC, arquitetura em camadas, Clean Architecture, Hexagonal, microservicos e o que aprender primeiro para vagas junior.

Resumo central:

Para começar bem: Controller -> Service -> Repository -> Database.

1. O que e arquitetura de projeto?

Arquitetura de software e a forma como voce organiza o sistema para que ele seja compreensivel, testavel e facil de evoluir. Em um projeto Spring Boot, arquitetura nao e apenas criar pastas bonitas. Ela define onde fica cada responsabilidade: quem recebe a requisicao, quem valida dados, quem executa regra de negocio, quem conversa com o banco e quem monta a resposta.

Uma boa arquitetura responde perguntas simples: se eu precisar alterar uma regra, onde mexo? Se eu trocar o banco, o impacto fica concentrado? Se der erro, consigo entender o caminho da requisicao? Se outro dev entrar no projeto, ele entende onde colocar um novo recurso?

Analogia: pense em um hospital. A recepcao recebe o paciente, o medico avalia, o laboratorio gera exames, a farmacia entrega medicamento e o sistema registra tudo. Se a recepcao tentasse diagnosticar, prescrever e registrar sozinha, viraria caos. No Spring, acontece o mesmo: Controller nao deve fazer tudo.

A ideia central e separar responsabilidades. Cada camada tem um papel claro e conversa com a proxima camada por contratos simples.

Resumo rapido: em APIs Spring Boot para mercado junior, a arquitetura mais comum e a arquitetura em camadas, geralmente organizada em Controller, Service, Repository, Entity, DTO, Exception e Config.

2. O que e uma API REST em Java/Spring?

API e uma forma de um sistema conversar com outro. Em vez de uma pessoa clicar em botoes, um sistema faz uma chamada HTTP para outro sistema. Uma API REST usa recursos, URLs, metodos HTTP e respostas padronizadas.

Metodo	Uso comum	Exemplo
GET	Buscar dados	GET /produtos
POST	Criar um novo registro	POST /produtos
PUT	Atualizar um registro inteiro	PUT /produtos/10
PATCH	Atualizar parte de um registro	PATCH /produtos/10/status
DELETE	Remover um registro	DELETE /produtos/10

No Spring Boot, uma API REST geralmente usa classes anotadas com **@RestController**. Essa classe recebe a requisicao, chama a camada de servico e devolve uma resposta HTTP com JSON.

```
@RestController
@RequestMapping("/produtos")
public class ProdutoController {

    private final ProdutoService service;

    public ProdutoController(ProdutoService service) {
        this.service = service;
    }

    @GetMapping
    public List<ProdutoResponseDTO> listar() {
        return service.listar();
    }
}
```

O controller acima nao sabe como o banco funciona. Ele so recebe a chamada HTTP e delega o trabalho para o service. Isso e importante porque evita que a entrada da aplicacao vire um lugar cheio de regras misturadas.

3. O que e MVC?

MVC significa Model, View e Controller. E um padrao classico para separar responsabilidades em aplicacoes web.

Parte	Funcao	Exemplo em aplicacao web
Model	Representa os dados e parte do dominio	Produto, Cliente, Pedido
View	Interface visual apresentada ao usuario	Tela HTML, Thymeleaf, React, Angular
Controller	Recebe interacoes/requisicoes e coordena o fluxo	ProdutoController

Em uma aplicacao Spring MVC tradicional com telas, o Controller pode retornar uma View. Exemplo: uma tela HTML renderizada pelo servidor. Mas em APIs REST, geralmente nao existe View no backend. A View fica em outro lugar: frontend web, app mobile, Postman, outro sistema ou uma automacao.

Por isso, quando falamos de Spring Boot API REST, o MVC costuma virar uma organizacao pratica em camadas: Controller, Service, Repository e Model/Entity.

```

Cliente HTTP/Postman/Frontend
  |
  v
Controller -> Service -> Repository -> Database
  |
  v
Resposta JSON

```

Analogia: o Controller e como o atendente de balcao. Ele entende o pedido do cliente e encaminha para quem resolve. O Service e o especialista que sabe a regra. O Repository e o setor que busca ou grava informacoes no arquivo/banco. O banco e o arquivo oficial.

4. Arquitetura em camadas no Spring Boot

Essa e a arquitetura que voce deve dominar primeiro. Ela aparece em muitos cursos, projetos internos de empresas, APIs corporativas e vagas junior.

Camada	Responsabilidade	O que evitar
Controller	Receber requisicoes HTTP, validar entrada basica, chamar service e devolver status/DTO.	Colocar regra de negocio pesada ou SQL.
Service	Concentrar regras de negocio, validacoes de fluxo, transacoes e orquestracao.	Retornar Entity diretamente sem necessidade ou fazer formatacao HTTP.
Repository	Acessar dados com Spring Data JPA, JPQL, SQL nativo ou queries derivadas.	Colocar regra de negocio.
Entity	Mapear tabela/objeto persistido pelo JPA.	Usar como contrato externo da API sem cuidado.
DTO	Transportar dados de entrada e saida da API.	Virar entidade com regra complexa.
Exception/Handler	Padronizar erros da API.	Espalhar try/catch repetido por todo controller.

Estrutura comum de pacotes:

```

src/main/java/com/empresa/projeto
  controller
  service
  repository
  entity
  dto
  exception
  config

```

Essa organizacao nao e uma lei universal, mas e excelente para aprender, trabalhar em time e manter clareza. O importante e manter responsabilidades bem definidas.

5. Exemplo completo: Produto API

Vamos imaginar uma API simples de produtos. O objetivo é cadastrar e listar produtos. Mesmo sendo simples, ela já mostra a arquitetura básica de um projeto real.

Entity

```
@Entity
@Table(name = "produtos")
public class Produto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;
    private BigDecimal preco;
    private Boolean ativo = true;

    // getters e setters
}
```

DTO de entrada

```
public record ProdutoRequestDTO(
    @NotBlank String nome,
    @NotNull @Positive BigDecimal preco
) {
}
```

DTO de saída

```
public record ProdutoResponseDTO(
    Long id,
    String nome,
    BigDecimal preco,
    Boolean ativo
) {
    public ProdutoResponseDTO(Produto produto) {
        this(produto.getId(), produto.getNome(), produto.getPreco(), produto.getAtivo());
    }
}
```

Repository

```
public interface ProdutoRepository extends JpaRepository<Produto, Long> {

    List<Produto> findByAtivoTrue();

    boolean existsByNomeIgnoreCase(String nome);
}
```

Service

```
@Service
public class ProdutoService {

    private final ProdutoRepository repository;

    public ProdutoService(ProdutoRepository repository) {
        this.repository = repository;
    }

    public ProdutoResponseDTO criar(ProdutoRequestDTO dto) {
        if (repository.existsByNomeIgnoreCase(dto.nome())) {
            throw new RegraNegocioException("Produto já cadastrado.");
        }

        Produto produto = new Produto();
        produto.setNome(dto.nome());
        produto.setPreco(dto.preco());
        produto.setAtivo(true);

        Produto salvo = repository.save(produto);
        return new ProdutoResponseDTO(salvo);
    }
}
```

```

    public List<ProdutoResponseDTO> listarAtivos() {
        return repository.findByAtivoTrue()
            .stream()
            .map(ProdutoResponseDTO::new)
            .toList();
    }
}

```

Controller

```

@RestController
@RequestMapping("/produtos")
public class ProdutoController {

    private final ProdutoService service;

    public ProdutoController(ProdutoService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<ProdutoResponseDTO> criar(@RequestBody @Valid
        ProdutoRequestDTO dto) {
        ProdutoResponseDTO response = service.criar(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @GetMapping
    public ResponseEntity<List<ProdutoResponseDTO>> listar() {
        return ResponseEntity.ok(service.listarAtivos());
    }
}

```

Observe que cada classe tem um motivo para existir. O Controller conhece HTTP. O Service conhece regra de negocio. O Repository conhece persistencia. A Entity representa a tabela. Os DTOs definem o contrato da API.

6. Por que usar DTO e nao expor Entity?

Entity e o modelo usado pelo JPA para conversar com o banco. DTO e o modelo usado pela API para conversar com o mundo externo. Misturar os dois pode parecer mais rapido no começo, mas cria problemas quando o projeto cresce.

Problema ao expor Entity	Exemplo
Vazamento de campos internos	A API pode devolver dataCriacao, usuarioAlteracao, flags internas ou campos sensiveis.
Acoplamento com o banco	Uma alteracao na tabela pode quebrar o contrato da API.
Relacionamentos infinitos	Cliente -> Pedido -> Cliente -> Pedido pode gerar loop de serializacao.
Dificuldade de validacao	Entrada e saida geralmente precisam de regras diferentes.

Analogia: a Entity e como o prontuario completo do paciente. O DTO e o resumo autorizado para cada finalidade. Nem todo mundo precisa, ou deve, ver tudo.

Boa pratica para junior: use DTO de entrada para dados recebidos e DTO de saida para dados retornados. Isso deixa sua API mais profissional.

7. Tratamento de erros em uma API

Uma API bem feita nao devolve erro baguncado. Ela responde com status HTTP correto e mensagem clara. No Spring, uma forma muito usada e criar um handler global com **@RestControllerAdvice**.

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RegraNegocioException.class)
    public ResponseEntity<ErroResponseDTO> tratarRegra(RegraNegocioException ex) {
        ErroResponseDTO erro = new ErroResponseDTO(

```

```

        "REGRA_NEGOCIO",
        ex.getMessage()
    );
    return ResponseEntity.badRequest().body(erro);
}
}

```

Status	Quando usar
200 OK	Busca ou operacao realizada com sucesso.
201 Created	Registro criado com sucesso.
400 Bad Request	Erro de validacao ou regra de negocio.
401 Unauthorized	Usuario nao autenticado.
403 Forbidden	Usuario autenticado, mas sem permissao.
404 Not Found	Registro nao encontrado.
500 Internal Server Error	Erro inesperado no servidor.

Para entrevistas e projetos, saber explicar isso conta muito: erro padronizado melhora consumo da API, debuggabilidade e experiencia de outros devs.

8. Tipos de arquitetura mais usados

Arquitetura	Ideia	Quando aparece
MVC	Separa Model, View e Controller.	Aplicacoes web e base conceitual do Spring MVC.
Camadas	Separa Controller, Service, Repository, Entity e DTO.	APIs REST corporativas e projetos junior/pleno.
Monolito	Um sistema unico com deploy unico.	Sistemas internos, ERPs, backends iniciais.
Monolito modular	Um monolito organizado por modulos de negocio.	Projetos medios/grandes que ainda nao precisam de microservicos.
Clean Architecture	Dominio e casos de uso ficam no centro, tecnologia fica nas bordas.	Projetos com regra de negocio mais forte.
Hexagonal	Ports and Adapters: entradas e saidas plugadas no nucleo.	Sistemas com muitas integracoes, filas, bancos ou APIs externas.
Microservicos	Varios servicos pequenos, cada um com responsabilidade propria.	Empresas maiores, escala, times separados.
Event-driven	Sistemas reagem a eventos por filas/mensageria.	Kafka, RabbitMQ, integracoes assincronas.

Todas sao importantes, mas nao devem ser estudadas com a mesma prioridade. Para uma vaga junior Java/Spring, dominar camadas, API REST, JPA, DTO, erros e testes e mais valioso do que tentar implementar microservicos complexos cedo demais.

9. Clean Architecture de forma simples

Clean Architecture tenta proteger a regra de negocio contra detalhes externos. O centro do sistema nao deveria depender diretamente de framework, banco, tela ou API externa. Esses detalhes ficam nas camadas de fora.

domain

```
entity
rule

application
usecase
port

infrastructure
persistence
externalapi

web
controller
dto
```

Exemplo: em vez de o caso de uso depender diretamente de `ProdutoRepository` do Spring Data JPA, ele depende de uma interface. A implementacao real fica na infraestrutura.

```
public interface ProdutoGateway {
    Produto salvar(Produto produto);
    boolean existePorNome(String nome);
}

public class CriarProdutoUseCase {

    private final ProdutoGateway gateway;

    public CriarProdutoUseCase(ProdutoGateway gateway) {
        this.gateway = gateway;
    }

    public Produto executar(String nome, BigDecimal preco) {
        if (gateway.existePorNome(nome)) {
            throw new RegraNegocioException("Produto ja cadastrado.");
        }
        return gateway.salvar(new Produto(nome, preco));
    }
}
```

Quando aprender: depois que voce estiver confortavel com Controller, Service, Repository, DTO, JPA e testes. Clean Architecture faz mais sentido quando voce ja sentiu a dor de projeto crescendo.

10. Arquitetura Hexagonal

A Arquitetura Hexagonal, tambem chamada de Ports and Adapters, coloca a regra de negocio no centro. As entradas e saidas ficam nas bordas: API REST, banco, fila, e-mail, API externa, arquivo etc.

```
Entrada HTTP
|
REST Adapter
|
Use Case / Domain
|
Port de saida
|
Repository Adapter
|
Database
```

Termo	Significado
Port	Interface que define o que o dominio precisa.
Adapter	Implementacao tecnica dessa interface.
Inbound adapter	Entrada do sistema, como Controller REST.
Outbound adapter	Saida do sistema, como banco, e-mail ou API externa.

Analogia: pense em uma tomada. O aparelho nao precisa saber toda a engenharia da rede eletrica. Ele precisa de uma porta padronizada. Se mudar a fonte de energia, voce adapta do lado de fora.

Essa arquitetura é excelente para sistemas com muitas integrações. No seu contexto de analista/dev, ela faz sentido para projetos que consomem APIs, geram PDF, enviam e-mail, consultam banco e usam filas.

11. Microserviços: aprender, mas no momento certo

Microserviços dividem um sistema em vários serviços independentes. Cada serviço tem uma responsabilidade clara, pode ter banco próprio e pode ser implantado separadamente.

```
servico-clientes
servico-produtos
servico-pedidos
servico-pagamentos
servico-notificacoes
```

Eles resolvem alguns problemas, mas criam outros: comunicação entre serviços, observabilidade, logs distribuídos, transações distribuídas, deploy, versionamento de contratos, autenticação entre serviços e monitoramento.

Por isso, para junior, microserviços devem ser estudados como conceito depois de uma base sólida. Não comece por eles. Primeiro aprenda a construir uma API monolítica bem feita.

Antes de microserviços, domine	Por que
API REST	Serviços conversam por contratos HTTP ou mensageria.
Banco de dados	Cada serviço precisa persistir e consultar dados corretamente.
Docker	Ajuda a rodar serviços, bancos e ambientes.
Testes	Mudanças entre serviços quebram fácil sem testes.
Logs e tratamento de erro	Debug distribuído é mais difícil.
Spring Security	Serviços reais precisam de segurança.

12. Monolito modular: muito útil no mercado

Muita gente pula direto de CRUD para microserviços, mas existe um meio-termo muito forte: monolito modular. É um único sistema, com um único deploy, mas organizado por módulos de negócio.

```
com.empresa.sistema
  produto
    ProdutoController
    ProdutoService
    ProdutoRepository
  pedido
    PedidoController
    PedidoService
    PedidoRepository
  cliente
    ClienteController
    ClienteService
    ClienteRepository
```

Esse modelo é excelente para aprender porque te obriga a pensar em limites de negócio sem trazer toda a complexidade de microserviços.

Em sistemas hospitalares, ERPs e sistemas internos, monolitos bem organizados ainda são extremamente comuns. O problema geralmente não é ser monolito; é ser um monolito sem organização.

13. Qual arquitetura você deve aprender primeiro?

Para seu objetivo de evoluir como dev Java/Spring junior, a ordem recomendada é esta:

Ordem	Tema	Objetivo
1	MVC e camadas	Entender Controller, Service, Repository, DTO e Entity.
2	API REST	Criar endpoints bem modelados, com status HTTP correto.
3	JPA/Hibernate	Mapear entidades, relacionamentos e queries.
4	Validacao e erros	Usar Bean Validation e GlobalExceptionHandler.
5	Testes	Testar service, repository e controller.
6	Spring Security	Autenticacao, autorizacao e JWT.
7	Monolito modular	Organizar projetos maiores por dominio.
8	Clean Architecture	Separar dominio de frameworks e banco.
9	Hexagonal	Trabalhar melhor com integracoes e adapters.
10	Microservicos	Aprender quando ja dominar APIs, banco, Docker e testes.

Minha recomendacao direta: faca projetos primeiro em arquitetura em camadas. Depois refatore um projeto pequeno para Clean/Hexagonal. Assim voce entende o motivo da arquitetura, nao apenas a forma.

14. O que um junior deve conseguir explicar em entrevista

Se voce souber responder os pontos abaixo com suas palavras, ja estara acima de muita gente que apenas copiou CRUD.

- Controller recebe HTTP e nao deve concentrar regra de negocio.
- Service executa regra de negocio e coordena o fluxo da operacao.
- Repository acessa o banco e abstrai a persistencia.
- DTO protege o contrato da API e evita expor Entity diretamente.
- Entity representa o modelo persistido pelo JPA.
- Bean Validation valida entrada antes da regra principal.
- GlobalExceptionHandler padroniza resposta de erro.
- Arquitetura em camadas e simples, clara e muito usada em APIs REST.
- Clean e Hexagonal protegem o dominio de detalhes tecnologicos.
- Microservicos nao sao automaticamente melhores; eles fazem sentido quando ha necessidade real de escala, autonomia e separacao de times.

Resposta curta boa para entrevista: em projetos Spring Boot eu costumo organizar a API em camadas. O Controller recebe as requisicoes, o Service concentra as regras de negocio, o Repository acessa o banco com Spring Data JPA, as Entities representam as tabelas e os DTOs definem a entrada e saida da API. Para erros, uso um handler global para padronizar respostas.

15. Checklist para um projeto Spring bem organizado

Item	Pergunta de verificacao
Pacotes	As classes estao organizadas por responsabilidade ou por modulo de negocio?
Controller	Tem apenas fluxo HTTP ou esta cheio de regra?

Item	Pergunta de verificacao
Service	As regras principais estao concentradas nele?
Repository	As queries estao claras e com nomes compreensiveis?
DTO	A API evita expor Entity diretamente?
Validacao	A entrada usa @Valid, @NotBlank, @NotNull, @Positive etc.?
Erros	Existe padrao de resposta para erros?
Transacao	Operacoes de escrita usam @Transactional quando necessario?
Testes	Ha testes para regras importantes?
README	O projeto explica como rodar e quais endpoints existem?

16. Mini projeto recomendado para treinar

Projeto: API de controle financeiro pessoal. Esse projeto treina CRUD, regras, DTO, JPA, validacao, tratamento de erro e filtros.

Modulo	O que implementar
Usuarios	Cadastro, busca e inativacao.
Categorias	Exemplo: Alimentacao, Transporte, Estudos, Saude.
Lancamentos	Receita ou despesa, valor, data, descricao e categoria.
Relatorios	Total por mes, total por categoria, saldo geral.
Regras	Nao permitir valor negativo, nao deletar categoria em uso, validar usuario ativo.
Extras	JWT, Pageable, filtros por periodo, Docker com PostgreSQL.

Comece em camadas. Depois, quando estiver pronto, tente separar por modulo: usuario, categoria, lancamento e relatorio. Em seguida, reescreva um caso de uso em estilo Clean Architecture para sentir a diferenca.

17. Referencias oficiais usadas

As referencias abaixo foram usadas como base conceitual para alinhar os termos ao ecossistema Spring:

- Spring Framework Javadoc - @Component e estereotipos como @Service, @Controller e @Repository.
- Spring Framework Reference Documentation - controllers anotados, mapeamento de requisicoes e Spring MVC.
- Spring Data JPA Reference Documentation - suporte a repositories baseados em JPA.
- Spring Guides - Building a RESTful Web Service e Building REST services with Spring.

Observacao: arquitetura nao e receita unica. O melhor desenho depende do tamanho do projeto, do time, das regras de negocio, das integracoes e da necessidade de evolucao. Para aprender e entrar no mercado, camadas bem feitas sao o melhor ponto de partida.